This technical report provides the detailed pseudocode of the proposed greedy algorithms and the proofs of the theoretical results. Algorithm 1 (Followers) describes the procedure for computing the follower set induced by an anchored edge. Algorithm 2 (GIA) presents the greedy incremental anchoring strategy, which iteratively selects the edge–increment action with the largest follower gain per unit budget. Algorithm 3 (GPA) accelerates GIA by using candidate reduction, follower-ratio upper bounds, and intermediate-result reuse, while preserving the same greedy selection criterion.

A Pseudo code of Followers

Algorithm 1: Followers(G, e, δ_e, s, C_s)

```
PQ ← queue(); // sort by coreness-layer ascending order
F ← ∅;
w(e) ← w(e) + δ_e;
for each x ∈ {u, v} if x ∉ C_s do
    PQ.add(x);
while PQ ≠ ∅ do
    x ← PQ.pop();
    w⁺(x) ← SupportWeightedDegree(G, x);
    if w⁺(x) ≥ s then
        F ← F ∪ {x};
        for each y ∈ N(x, G) and x ≺ y do
            if y ∉ C_s and y ∉ F then
                PQ.add(y);
    else
        Q ← queue();
        Q.push(x);
        while Q ≠ ∅ do
            y ← Q.pop();
            if y ∈ F then
                F ← F \ {y};
            for each z ∈ N(y, G) ∩ F do
                w⁺(z) ← w⁺(z) − w(y, z);
                if w⁺(z) < s then
                    Q.push(z);
w(e) ← w(e) − δ_e;
return F
```

Algorithm description. Algorithm 1 computes the follower set induced by an anchored edge via a localised simulation of the s-core peeling process. Starting from the anchored endpoints, the algorithm explores only nodes that are reachable through neighbours with higher coreness–layer pairs, following the upstair order established by the onion-layer structure. A node is retained as a follower if its support weighted degree meets the s-core threshold; otherwise, it is discarded. Discarding a node may reduce the support of neighbouring retained followers, potentially triggering further removals. This cascading update exactly mirrors the behaviour of the s-core decomposition, but is restricted to the structurally affected region. The process terminates when no further admissible nodes remain, and the surviving candidates constitute the final follower set.

B Pseudo code of GIA

Algorithm description. Algorithm 2 presents the baseline greedy incremental anchoring procedure of GIA. Starting from the initial s-core, the algorithm iteratively evaluates candidate edges whose endpoints are not both contained in the current s-core. For each candidate edge, GIA determines the required weight increment using the

Algorithm 2: GIA(G, b, s)

```
G' ← G, A ← ∅, sum ← 0;
C_s ← s-core(G');
while sum < b do
    E* ← (V 2) \ (C_s 2);
    e* ← none, δ* ← 0, max_FR ← 0;
    for each e ∈ E* do
        δ_e ← computeDelta(G', s, e);
        if sum + δ_e ≤ b then
            F(e, δ_e) ← Followers(G', e, δ_e, s, C_s);
            FR(e) ← |F(e, δ_e)|/δ_e;
            if max_FR < FR(e) then
                e* ← e, δ* ← δ_e;
                max_FR ← FR(e);
    w_G'(e*) ← w_G'(e*) + δ*;
    sum ← sum + δ*;
    A ← A ∪ (e*, δ*);
    C_s ← s-core(G');
return A
```

CGI rule, computes the induced follower set, and evaluates the corresponding follower ratio. At each iteration, the edge achieving the largest follower ratio is selected, its weight is increased accordingly, and the s-core is updated. This process repeats until the available budget is exhausted, yielding a greedy baseline for comparison.

C Pseudo code of GPA

Algorithm description. Algorithm 3 presents the pseudocode of GPA, an accelerated variant of GIA that reduces the candidate search space while preserving the same greedy selection criterion. GPA exploits the locality of follower propagation by organising nodes outside the s-core into connected components and maintaining, for each component, the best edge–weight increment candidate pair identified so far. At each iteration, GPA selects the current best candidate across all components. It then explores candidate edges whose endpoints belong to different components, guided by node-level and edge-level upper bounds that allow early termination and safe pruning. Only candidate pairs that cannot be ruled out by these bounds are evaluated using the same follower computation as in GIA. After applying the selected edge reinforcement, GPA updates the component structure. If the reinforced edge connects two distinct components, they are merged and their best candidate is recomputed; otherwise, only the affected component is updated. In both cases, the s-core is recomputed locally within the affected components. This process repeats until the budget is exhausted, yielding the same greedy outcome as GIA with fewer candidate evaluations.

D Proof of Theorem 1

Proof. We reduce the Maximum Coverage (MC) problem, which is known to be NP-hard unless P = NP [2], to the WASCO problem. The MC problem is defined as follows: given a collection of sets $\{T_1, T_2, \ldots, T_t\}$ containing elements from a universe $\{e_1, e_2, \ldots, e_d\}$

Algorithm 3: GPA(G, b, s)

```
G' ← G, A ← ∅, sum ← 0;
L ← list(), M ← map(); // maps each component to its best (e, δ_e, FR)
C_s ← s-core(G');
M ← buildComponents(G',b,s);
while sum < b do
    e*, δ*, max_FR ← arg max_{(e,δ_e,FR)∈M} FR;
    for u ∈ V \ C_s and sum + (s − c(u)) ≤ b do
        U(u) ← UpperBound(G', u, s);
        L.append(u);
    sort(L); // by upper bound decreasing order
    for u ∈ L do
        if max_FR > 2 · U(u) then
            break;
        for v ∈ L s.t. v appears after u do
            if u.component = v.component then
                continue;
            if max_FR > U(u) + U(v) then
                break;
            if max_FR > U(u, v) then
                continue;
            else
                e ← (u, v);
                δ_e ← computeDelta(G', s, e);
                if sum + δ_e ≤ b then
                    F(e, δ_e) ← Followers(G', e, δ_e, s, C_s);
                    FR(e) ← |F(e, δ_e)|/δ_e;
                    if max_FR < FR(e) then
                        e* ← e, δ* ← δ_e;
                        max_FR ← FR(e);
    w_{G'}(e*) ← w_{G'}(e*) + δ*;
    sum ← sum + δ*;
    A ← A ∪ (e*, δ*);
    updateComponents(G',b,s,sum,M);
return A
```

and a budget b', the objective is to select b' sets such that the number of unique elements covered is maximised.

To construct an instance of the WASCO problem, consider an arbitrary instance I_{MC} of the MC problem with sets $\{T_1, \ldots, T_t\}$ and elements $\{e_1, \ldots, e_d\} = \bigcup_{i=1}^{t} T_i$, as illustrated in the top of Figure 1. We define an instance $I_{\text{WASCO}} = (G, s, b)$ as follows:

- The graph G consists of three components: a part M representing the sets T_i, a part N representing the elements e_j, and a set of $(d+2)$-cliques.
- For each set T_i, create a corresponding set of nodes M_i in M, where $M_i = \{u_{i,1}, u_{i,2}, \ldots, u_{i,d+2}\}$. The nodes $\{u_{i,1}, \ldots, u_{i,d+1}\}$ form a $(d+1)$-clique, and $u_{i,d+2}$ is connected to $\{u_{i,1}, \ldots, u_{i,d}\}$.
- For each element e_j, create a corresponding node v_j in N. Add an edge between v_j and $u_{i,j}$ for every set T_i such that $e_j \in T_i$.

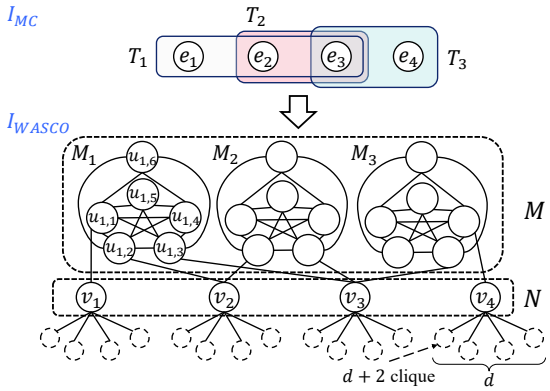


Figure 1: Proof of NP-hardness

- For each v_j, add d disjoint $(d+2)$-cliques, and connect v_j to one node in each clique.
- Set $s = d + 1$ and $b = b'$.
- Every edge in G has a weight of 1.

This construction can clearly be completed in polynomial time. Initially, during the s-core decomposition, all nodes in M and N are removed, leaving only the $(d+2)$-cliques. This happens because:

- Nodes $u_{i,d+1}$ and $u_{i,d+2}$ in M_i have degree d, which is less than $s = d + 1$, and are therefore removed.
- Removing $u_{i,d+1}$ and $u_{i,d+2}$ causes the remaining nodes $u_{i,j}$ in M_i to fail the s-core condition, as their degree drops below s.
- All v_j nodes in N are similarly removed because their connections to M_i are lost.

As a result, only the $(d+2)$-cliques remain in the s-core after this initial decomposition. To maximise the s-core size, an optimal solution is to add an edge between $u_{i,d+1}$ and $u_{i,d+2}$ in M_i. Adding this edge corresponds to selecting the set T_i in the MC problem. Note that another optimal solution adds edges between distinct sets in M (e.g., $(u_{i,d+2}, u_{j,d+1})$ and $(u_{j,d+2}, u_{i,d+1})$), which can always be replaced by the intra-set connection since both strategies yield the exact same number of followers. When the edge $(u_{i,d+1}, u_{i,d+2})$ is added, this single edge brings the entire M_i together with all of its adjacent v_j nodes in N into the s-core:

- All nodes in M_i are included, since the degrees of $u_{i,d+1}$ and $u_{i,d+2}$ become $d + 1$.
- All v_j connected to M_i are included as well, as their degree conditions are now satisfied.

We now argue that this connection is optimal. Our strategy gains an entire M_i and its adjacent v_j nodes at the cost of a single action, so to outperform it, an alternative must achieve a larger gain at the same cost. We therefore compare every candidate from the viewpoint of the gain a single weight increment produces. Recall that M_i survives if and only if *both* $u_{i,d+1}$ and $u_{i,d+2}$ reach a weighted degree of $d + 1$; otherwise, once either is removed, every remaining node $u_{i,j}$ with $j \leq d$ also drops below s and the whole M_i collapses. We thus examine every way of spending one unit of weight:

- **Raising the weight of an existing edge from $u_{i,d+1}$ or $u_{i,d+2}$ to a node in $\{u_{i,1}, \ldots, u_{i,d}\}$ by 1.** This lifts only one of the two deficient nodes to weighted degree $d + 1$, so the other still drops out and all of M_i collapses. The gain is 0.

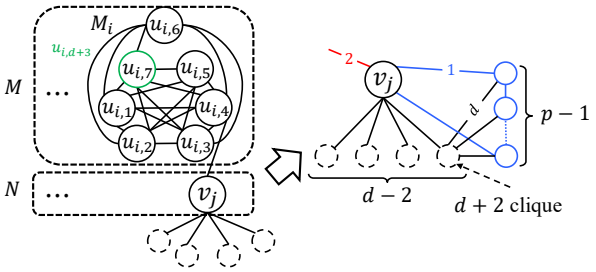


Figure 2: Gadget for the inapproximability reduction

- **Cross connections between distinct M_i and M_j** (e.g., adding weight on $(u_{i,d+2}, u_{j,d+1})$ and $(u_{j,d+2}, u_{i,d+1})$). This can revive both M_i and M_j, but it spends two units—exactly the same budget as the two intra connections $(u_{i,d+1}, u_{i,d+2})$ and $(u_{j,d+1}, u_{j,d+2})$—and revives the same M_i, hence the same v_j nodes. It can therefore always be replaced by intra connections with no loss. Cross connections to a node in $\{u_{\cdot,1}, \ldots, u_{\cdot,d}\}$ are inferior for the same reason as the first case.
- **Adding weight within N** (i.e., on (v_j, v_k)). Since each v_j is already connected to d of the $(d+2)$-cliques, it only needs one more unit of weighted degree to survive; thus a single unit on (v_j, v_k) can bring *two* nodes v_j and v_k into the s-core. However, this gain of 2 is far smaller than the gain of our strategy, so it is not optimal.
- **Connecting a node in N to $u_{i,d+1}$ or $u_{i,d+2}$.** Again only one of the two deficient nodes is filled, so the other collapses and M_i cannot survive. The gain is 0.
- **Adding weight to an edge involving a node in the $(d+2)$-cliques.** By the same mechanism, this fills at most one deficient node, which is insufficient to revive any M_i. The gain is 0.

Therefore, among all candidates, spending one unit of weight on the connection $(u_{i,d+1}, u_{i,d+2})$ yields the largest gain, so an optimal solution spends each of its b units to complete a distinct M_i. This corresponds exactly to selecting the set T_i in the MC problem. The resulting s-core size gain equals

$$\underbrace{(d+2)\cdot b}_{\text{nodes of the selected } M_i} + \left|\{\, v_j : e_j \in \textstyle\bigcup_{i\in S} T_i \,\}\right|,$$

where S is the set of selected M_i with $|S| = b$. The first term is constant for any solution using the full budget, while the second counts the distinct elements covered. Maximising the s-core size thus reduces exactly to maximising the number of distinct covered elements—the objective of the MC problem.

Since the optimal way to maximise the s-core is to add edges corresponding to specific T_i sets so as to include as many e_i elements as possible in the s-core, this process directly mirrors the objective of the MC problem to select sets that maximise the coverage of elements. Thus, selecting M_i that are more densely connected to uncovered v_j aligns with the optimisation goals of the MC problem.

Therefore, the MC problem is reducible to the WASCO problem, implying that $I_{MC} \leq_P I_{\text{WASCO}}$. As the MC problem is NP-hard, this proves the NP-hardness of the WASCO problem. □

E Proof of Theorem 2

PROOF. We give a gap-preserving reduction from the Maximum Coverage (MC) problem. It is known that, for any $\varepsilon > 0$, MC cannot be approximated within a factor of $(1 - 1/e + \varepsilon)$ in polynomial time, unless P = NP [1].

We start from the construction in the proof of Theorem 1 and modify both the set gadget M and the element part N (Figure 2).

Modification of the set gadget M. For each set gadget M_i, we add one new node $u_{i,d+3}$ and connect it to all of $\{u_{i,1}, \ldots, u_{i,d+2}\}$, so that its degree is $d+2$. Crucially, $u_{i,d+3}$ is *not* connected to any element node v_j. We also raise the threshold to $s = d+2$.

Modification of the element part N. For each element node v_j, we (i) reduce the number of disjoint $(d+2)$-cliques attached to v_j from d to $d-2$, (ii) replace v_j by a cycle on p nodes by adding $p-1$ new nodes connected to v_j with edges of weight 1, where each of the $p-1$ new cycle nodes is connected to one node of a (randomly chosen) attached $(d+2)$-clique by an edge of weight d, and (iii) set the weight of every edge $(v_j, u_{i,j})$ between an element node and its set gadget to 2. Let p be an integer chosen as a polynomially bounded function of the MC instance size; its value will be specified at the end of the proof.

Let G be the resulting WASCO instance, and let $f(A) = |C_s(G_A)|$ denote the size of the s-core after applying an anchoring solution A with total budget b. As in Theorem 1, without any anchoring, all nodes in M and N are removed by the s-core decomposition, and only the union of the $(d+2)$-cliques remains in the s-core. To see that the new node $u_{i,d+3}$ is also removed, note that once $u_{i,d+1}$ and $u_{i,d+2}$ drop below $s = d+2$ and are removed, the degree of $u_{i,d+3}$ falls from $d+2$ to $d < s$, so it is removed as well. Denote by B the number of nodes in the always-retained clique part; B is fully determined by the MC instance and is independent of A.

Now consider any feasible anchoring solution A that uses exactly b budget units (if it uses fewer, we can add arbitrary weight increments on edges between already-retained clique nodes to exhaust the budget without changing the s-core size). By the same argument as in Theorem 1, anchoring a single intra-set edge $(u_{i,d+1}, u_{i,d+2})$ with weight 1 raises both endpoints to weighted degree $d+2 = s$; the auxiliary node $u_{i,d+3}$ then regains weighted degree $d+2 = s$, and consequently all $d+3$ nodes of M_i enter the s-core. This corresponds to selecting the set T_i in the MC problem; since exactly b sets are selected, this contributes an additive term $(d+3)b$ to $f(A)$.

We now argue that, as in Theorem 1, an optimal solution spends each budget unit on a distinct intra-set edge $(u_{i,d+1}, u_{i,d+2})$. The only alternative route to bring element nodes into the s-core is to reinforce edges inside N. Since each element node v_j has weighted degree d (from its $d-2$ clique edges and its two cycle neighbours) and $s = d+2$, it is short by 2. Because WASCO permits reinforcing any node pair, one may add weight 2 on a pair (v_j, v_k), lifting both to $d+2 = s$ simultaneously; this brings two element nodes (and their two cycles) into the s-core at a cost of 2 budget units. In contrast, spending 1 budget unit on an intra-set edge revives the entire M_i together with all element nodes attached to M_i. Even in the extreme case where each M_i contains a single element node, the set route brings the $d+3$ nodes of M_i into the s-core in addition to the element node and its cycle, whereas the N-internal route only obtains element

nodes and their cycles at twice the cost; hence the set route always dominates.

Let $\mathrm{cov}(A)$ be the number of elements covered by the corresponding b selected sets in the MC instance. By construction, $\mathrm{cov}(A)$ is exactly the number of element nodes v_j that enter the s-core under A. We next show that if a node v_j enters the s-core, then all p nodes on the cycle containing v_j also enter the s-core. Indeed, once v_j is retained, every other cycle node has weighted degree $d+2$ from its two cycle neighbours (weight 1 each) and its incident edge of weight d to an always-retained clique node. With the additional support propagated along the retained cycle, each cycle node satisfies the s-core condition in the induced subgraph consisting of the always-retained cliques together with this cycle. Therefore, the inclusion of v_j contributes an additional p nodes to the s-core.

Putting the above together, we obtain the following exact relation for every feasible optimal A:

$$f(A) = B + (d+3)b + p \cdot \mathrm{cov}(A).$$

Let $\mathrm{OPT}_{\mathrm{MC}}$ be the optimal MC value, and let $\mathrm{OPT}_{\mathrm{W}}$ be the optimal WASCO objective. From the relation above,

$$\mathrm{OPT}_{\mathrm{W}} = B + (d+3)b + p \cdot \mathrm{OPT}_{\mathrm{MC}}.$$

Assume, for contradiction, that there exists a polynomial-time $(1 - 1/e + \varepsilon)$-approximation algorithm for WASCO. Let $\rho = 1 - 1/e + \varepsilon$, and let A be the solution returned by this algorithm. Then

$$B+(d+3)b+p\cdot\mathrm{cov}(A) = f(A) \geq \rho\,\mathrm{OPT}_{\mathrm{W}} = \rho\big(B+(d+3)b+p\cdot\mathrm{OPT}_{\mathrm{MC}}\big),$$

which implies

$$\mathrm{cov}(A) \geq \rho\,\mathrm{OPT}_{\mathrm{MC}} - \frac{(1-\rho)\big(B+(d+3)b\big)}{p}.$$

Since $\rho \geq 0$, we have $1 - \rho \leq 1$, so the subtracted term is at most $\big(B + (d+3)b\big)/p$. Choosing

$$p \;\geq\; \frac{2\big(B+(d+3)b\big)}{\varepsilon},$$

which is polynomially bounded in the instance size, makes this term at most $\varepsilon/2$. Hence

$$\mathrm{cov}(A) \;\geq\; \rho\,\mathrm{OPT}_{\mathrm{MC}} - \frac{\varepsilon}{2}.$$

Since $\mathrm{OPT}_{\mathrm{MC}} \geq 1$ for any non-trivial MC instance, we have $\varepsilon/2 \leq (\varepsilon/2)\,\mathrm{OPT}_{\mathrm{MC}}$, and therefore

$$\mathrm{cov}(A) \;\geq\; \Big(\rho - \frac{\varepsilon}{2}\Big)\mathrm{OPT}_{\mathrm{MC}} \;=\; \Big(1 - 1/e + \frac{\varepsilon}{2}\Big)\mathrm{OPT}_{\mathrm{MC}}.$$

This yields a polynomial-time $(1 - 1/e + \varepsilon/2)$-approximation for MC, contradicting the inapproximability of MC unless P = NP [1]. Therefore, WASCO cannot be approximated within a factor of $(1 - 1/e + \varepsilon)$ in polynomial time for any $\varepsilon > 0$, unless P = NP. □

F Proof of Theorem 3

Proof. Suppose that f is submodular. Then, for any two sets A and B, it must hold that $f(A)+f(B) \geq f(A\cup B)+f(A\cap B)$. Consider a graph consisting of four nodes $\{u, v, x, y\}$ and two edges (u, x) and (v, y), each with a weight of 1. Let $s = 2$, if $A = \{\langle(u, v), 1\rangle\}$ and $B = \{\langle(x, y), 1\rangle\}$, $f(A) + f(B) = 0 < f(A \cup B) + f(A \cap B) = 4$. □

G Proof of Property 1

Ties in follower ratio are broken by a fixed total order $\prec_{\mathrm{tb}}$ on node pairs (e.g., lexicographic order of vertex identifiers), chosen independently of candidate enumeration. Both GIA and GPA use $\prec_{\mathrm{tb}}$, so the greedy winner in any fixed graph state is unique.

Proof. We show by induction over greedy iterations that GPA and GIA select the same node pair with the same weight increment at every iteration.

Base case. Both algorithms start from the same input (G, b, s), compute the same initial s-core $C_s(G)$, the same coreness–layer order, and the same CGI value for every candidate edge. Hence they enter the first iteration with the same state.

Inductive step. Assume that the two algorithms have selected identical edge–increment actions through the first $t-1$ iterations. Then they enter iteration t with the same current graph G_t, the same s-core $C_s(G_t)$, the same coreness–layer order, the same remaining budget, and the same CGI value for every candidate edge. Let e^* be the node pair selected by GIA in this state under $\prec_{\mathrm{tb}}$. We show that GPA also selects e^* with the same increment δ_{e^*}.

(1) **Candidate reduction.** If both endpoints of e^* lie outside $C_s(G_t)$, e^* is unaffected and retained. Otherwise, write $e^* = (u, v)$ with $u \notin C_s(G_t)$, $v \in C_s(G_t)$. By Theorem 5, every candidate (u, x) with $x \in C_s(G_t)$ shares the CGI value, follower set, and hence follower ratio of (u, v), since these depend only on the non-core endpoint u. These candidates form one tie-equivalent class. Because GPA keeps the representative of this class according to the same tie-breaking order $\prec_{\mathrm{tb}}$, the retained representative is the same node pair that GIA would select from this class. Therefore candidate reduction does not alter the greedy choice.

(2) **Upper-bound pruning.** GPA prunes a candidate only when the current best ratio ρ *strictly* exceeds a valid upper bound on its follower ratio. The candidate-skipping, partner-level termination, and global termination rules use the bounds $U(u, v)$, $U(u)+U(v)$, and $2U(u)$, respectively; by Definition 13 and the descending order of $U(\cdot)$, each bound is valid for every candidate it discards. If e^* were pruned with bound $B(e^*)$, then $FR(e^*) \leq B(e^*) < \rho$, meaning GPA already evaluated a candidate beating e^*—contradicting that e^* is the greedy winner. Hence e^* is never pruned. Moreover, because the pruning condition uses a strict inequality, candidates tying the current best are retained, so the common tie-breaking order remains applicable.

(3) **Intermediate-result reuse.** By Theorem 4, every follower of an anchored edge is reachable from one of its non-core endpoints by an upstair path, so newly activated nodes stay within the non-core components incident to that edge. A component K outside this region is unchanged: a node $x \in K$ could only change its support, coreness–layer pair, or upstair reachability by becoming adjacent to a newly activated node, but such a node was non-core just before reinforcement and would then have shared x's component—a contradiction. GPA thus reuses a component's cached best only when the component is unaffected and the cached candidate is still feasible under the remaining budget; otherwise it recomputes.

We track e^* under this scheme, in the two ways its endpoints can lie. If both endpoints of e^* lie in the same non-core component, then e^* is an intra-component candidate of that component. If

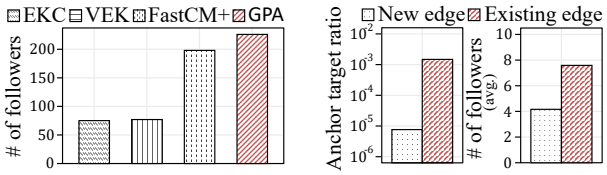
the component is recomputed, e^* is evaluated directly. If its cached best is reused, then since the component is unchanged every local candidate keeps its CGI value, follower ratio, and $\prec_{\text{tb}}$-priority, and any internal tie is resolved by the same $\prec_{\text{tb}}$; hence the cached best is exactly the pair GIA would select there, namely e^*. Either way e^* reaches the final comparison.

If the endpoints of e^* lie in different components, then e^* is an inter-component candidate, which reuse never caches away: it is freshly evaluated and enters the final comparison directly.

In that comparison, GPA weighs the cached intra-component bests against the freshly evaluated inter-component candidates under the same follower-ratio criterion, the same upper-bound pruning, and the same $\prec_{\text{tb}}$. This is precisely the setting of case (ii): e^* is the greedy winner, so it is never pruned and, on any tie, wins under $\prec_{\text{tb}}$. Therefore e^* is present and selected.

In every case e^* survives and is selected with the same increment δ_{e^*}, so both algorithms enter iteration $t+1$ in the same state. By induction, GPA and GIA select identically at every iteration and return the same final s-core. □

H Additional Experiment



(a) $k = 20, b = 200$ **(b) Anchored edge analysis**

Figure 3: EQ1-4. Effectiveness on the Facebook

EQ1-4. Effectiveness (unweighted graph). We further evaluate GPA on the unweighted Facebook dataset [3] ($|V| = 4,039$, $|E| = 88,234$, $d_{\text{avg}} = 43.7$). All compared methods, including EKC [5], VEK [6], and FastCM+ [4], are originally designed for unweighted graphs. To enable a fair comparison, we treat the unweighted graph as a special case of our weighted formulation by assigning unit weight to every edge, and follow the same parameter settings ($k = 20$, $b = 200$) as in FastCM+. As shown in Figure 3(a), GPA can achieve the largest number of followers among all compared methods under the same budget. This result indicates that, even when restricted to the unweighted setting, GPA is capable of operating effectively in practice. Figure 3(b) further analyses the anchoring behaviour of GPA. After normalising by the size of the corresponding feasible action spaces, edge reinforcement is selected more frequently than new edge insertion. Moreover, reinforcement actions tend to yield a higher average number of followers per anchored edge. These observations suggest that strengthening existing connections can be a more effective anchoring strategy than edge insertion, even when all edges have uniform weight.

References

[1] Uriel Feige. 1998. A threshold of ln n for approximating set cover. *Journal of the ACM (JACM)* 45, 4 (1998), 634–652.

[2] Samir Khuller, Anna Moss, and Joseph Seffi Naor. 1999. The budgeted maximum coverage problem. *Information processing letters* 70, 1 (1999), 39–45.

[3] Jure Leskovec and Julian Mcauley. 2012. Learning to discover social circles in ego networks. *Advances in neural information processing systems* 25 (2012).

[4] Xin Sun, Xin Huang, and Di Jin. 2022. Fast algorithms for core maximization on large graphs. *Proceedings of the Very Large Data Bases (VLDB) Endowment* 15, 7 (2022), 1350–1362.

[5] Zhongxin Zhou, Fan Zhang, Xuemin Lin, Wenjie Zhang, and Chen Chen. 2019. k-Core Maximization: An Edge Addition Approach.. In *International Joint Conference on Artificial Intelligence (IJCAI)*. 4867–4873.

[6] Zhongxin Zhou, Wenchao Zhang, Fan Zhang, Deming Chu, and Binghao Li. 2022. VEK: a vertex-oriented approach for edge k-core problem. *World Wide Web* 25, 2 (2022), 723–740.